

Insertion-Deletion & Predicate Reads (Phantoms) -- DBMS

Definition: The Phantom Phenomenon occurs when a concurrent transaction inserts or deletes tuples satisfying a search predicate while another transaction is executing range queries over the same predicate.

1. Detailed Technical Explanation

1. The Phantom Problem Scenario

Suppose Transaction T₁ reads all employees in Dept_ID = 10 twice within its execution:

Transaction T1	Transaction T2
-----	-----
Select * from Emp where Dept_ID = 10; (Returns 5 rows)	
	Insert into Emp values (106, 'Raj', 10); Commit;
Select * from Emp where Dept_ID = 10; (Returns 6 rows! Phantom Row detected!)	

Standard tuple-level locking fails to prevent this because the new tuple 106 did not exist when T₁ acquired its locks!

2. Solutions to Phantom Phenomenon

1. Predicate Locking (Logical Locking)

- * Transaction T₁ acquires a lock on a predicate condition (e.g., Dept_ID = 10).

2. Index Locking (Physical Granularity Solution)

- * If an index exists on Dept_ID, T₁ locks the index bucket / page corresponding to Dept_ID = 10.

3. Next-Key Locking (B+ Tree Index Lock)

- * Locks both the target index record AND the gap immediately preceding/following it in the B+ Tree index, preventing insertions into the range.

3. Core Concepts & Memory Keywords

- * Phantom Tuple: A newly inserted row appearing in a repeated range query.

4. Must-Write Points for Exams

- * Standard tuple locks cannot prevent phantom rows because phantom tuples do not exist at lock request time.

5. Quick Recall Flow

Range Query -> Concurrent Insert -> Phantom Row Appears -> Resolve via Predicate Locks or Next-Key Index Locks